# 📜 15712 Project

## Research Question

Can a general-purpose, format-aware compression framework (OpenZL), through automated, schema-driven plan generation, achieve compression ratios and computational overhead comparable to the specialized, manually-optimized otel-arrow protocol for OTLP data transport? More broadly, can a format-aware compression framework, when applied to a row-oriented protocol, achieve compression efficiency and transport performance comparable to that of a native columnar protocol?

## Evaluation

| Workload | Protocol | Compressor | Compression Rate |
|---|---|---|---|
| OpenTelemetry | Pure gRPC (OTLP) | zstd | A |
| OpenTelemetry | Pure gRPC (OTLP) | OpenZL | B |
| OpenTelemetry | Arrow IPC on gRPC (OTAP) | zstd | C |
| OpenTelemetry | Arrow IPC on gRPC (OTAP) | OpenZL | D |

## Expected Outcome

B > A (OpenZL > zstd)
D > C (OpenZL > zstd)
C > A (Columnar format helps improving compression rate)
B ~= D (Format-aware compressor is insensitive to whether the data is row based or column based)

## Evaluate another workload to backup our claim

Criteria:

(1) The system must have an established, real-world, row-oriented transport protocol and a corresponding specialized, columnar-native transport. This provides the "control" and "gold standard" for the comparison.
(2) The data stream is supposed to be sent over network and high-volume. The computational overhead and compression benefits of columnar formats are only

apparent when processing large batches of data (e.g., thousands or millions of records per second), not single, low-latency events.

(3) Has a well-defined and stable schema. The data must be structured (like Protobuf, Avro, or consistent JSON) rather than unstructured (like free-text logs). OpenZL's trainer requires a stable schema to analyze and generate its optimized compression plan.

Candidates:

(1) ClickHouse Read/Write
    (a) Avro vs Arrow?

## More references to cite

(1) Schema-Aware Compression for Structured Data (JSON/Logs)
(2) AoS vs. SoA (Array of Structures vs. Structure of Arrays)

## Implementation

https://github.com/danielthank/otel-arrow/tree/openzl (openzl branch)

# ⏳ Timeline

- Nov 3-9
    - Evaluate A, B, C, D in opentelemetry
- Nov 10-16
    - Another system
- Nov 17-23
    - Grpc vs. arrow for TPC-H/taxi
    - OTel
        - Batch size
        - Cpu, memory
        - Traces, logs
- Nov 24-30
    - Extended analysis, paper writing, presentation prep.

# 🔬 Experiment Result

## Hipstershop Metrics (Batch = 50000)

| | Uncompressed | | | Zstd (level 9) | | | OpenZL | | |
|---|---|---|---|---|---|---|---|---|---|
| | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt |
| OTLP | 37.17 MB | 1.00x | 556.3 | 464.93 KB | 79.94x | 7.0 | 375.67 KB | 98.94x | 5.6 |
| OTAP | 5.71 MB | 6.50x | 85.5 | 220.86 KB | 168.29x | 3.3 | 221.35 KB | 167.91x | 3.3 |

## Hipstershop Traces (Batch = 50000)

| | Uncompressed | | | Zstd (level 9) | | | OpenZL | | |
|---|---|---|---|---|---|---|---|---|---|
| | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt |
| OTLP | 88.98 MB | 1.00x | 1027.6 | 2.89 MB | 30.80x | 33.4 | 2.74 MB | 32.47x | 31.7 |
| OTAP | 8.83 MB | 10.08x | 102.0 | 1.78 MB | 50.01x | 20.5 | 1.77 MB | 50.15x | 20.5 |

## Astronomy Metrics (Batch = 50000)

| | Uncompressed | | | Zstd (level 9) | | | OpenZL | | |
|---|---|---|---|---|---|---|---|---|---|
| | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt |
| OTLP | 299.65 MB | 1.00x | 380.9 | 6.26 MB | 47.87x | 8.0 | 4.49 MB | 66.78x | 5.7 |
| OTAP | 64.79 MB | 4.62x | 82.4 | 3.31 MB | 90.66x | 4.2 | 3.26 MB | 91.92x | 4.1 |

## Astronomy Traces (Batch = 50000)

| | Uncompressed | | | Zstd (level 9) | | | OpenZL | | |
|---|---|---|---|---|---|---|---|---|---|
| | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt |
| OTLP | 62.84 MB | 1.00x | 979.6 | 2.95 MB | 21.27x | 46.1 | 2.54 MB | 24.70x | 39.7 |
| OTAP | 13.63 | 4.61x | 212.4 | 2.23 MB | 28.19x | 34.7 | 2.23 MB | 28.12x | 34.8 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | MB | | | | | | | | |

## TPC-H LineItem (Batch = 50000)

| | Uncompressed | | | Zstd (level 9) | | | OpenZL | | |
|---|---|---|---|---|---|---|---|---|---|
| | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt | Bytes | Ratio | Byte/pt |
| Proto | 84.56 MB | 1.00x | 140.8 | 22.72 MB | 3.72x | 37.8 | 17.86 MB | 4.73x | 29.7 |
| Arrow | 82.01 MB | 1.03x | 136.5 | 18.93 MB | 4.47x | 31.5 | - | - | - |

## OpenZL Severely Struggles with Single Data Points

## OTel ARROW + OpenZL is the Clear Winner for Large Batches

On the two large datasets:
- hipstershop: 3.2 bytes/point (99x compression ratio)
- hostandcollector: 0.9 bytes/point (115x compression ratio!)

## For Small Batches, Simple Zstd is More Reliable

On batch size = 1:
- OTLP + Zstd gives consistent ~25-65 bytes/point
- OTel ARROW + Zstd ranges from 40-383 bytes/point (worse overhead)
- OpenZL formats have massive overhead making them impractical

## Interesting Question:

What's the minimum batch size where OpenZL becomes worthwhile? The data shows batch size = 1 is terrible, but regular batches work great. It would be interesting to test batch sizes of 10, 50, 100, 500 to find the crossover point where OpenZL's overhead is amortized by its superior compression.

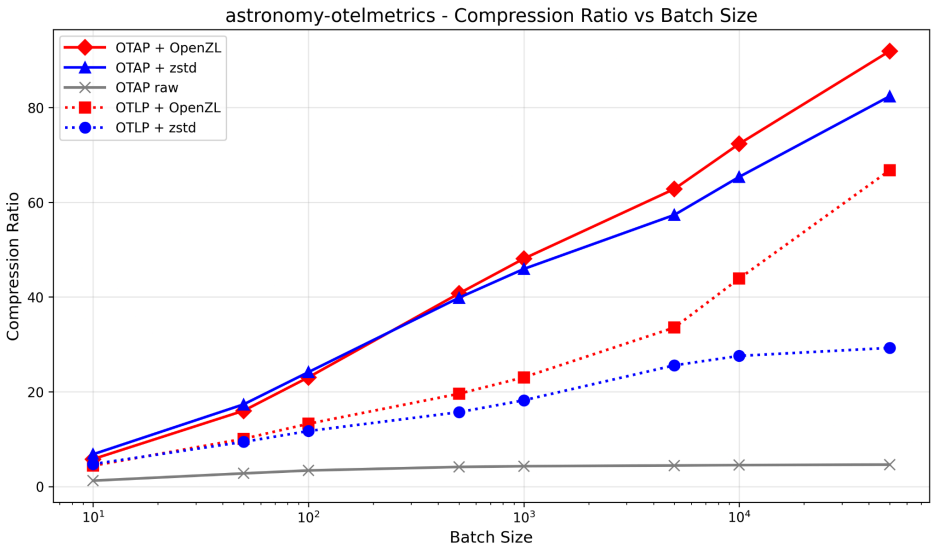
Groups of columns

TPC-H row
Strings and number

# Experiment Result (Batch)

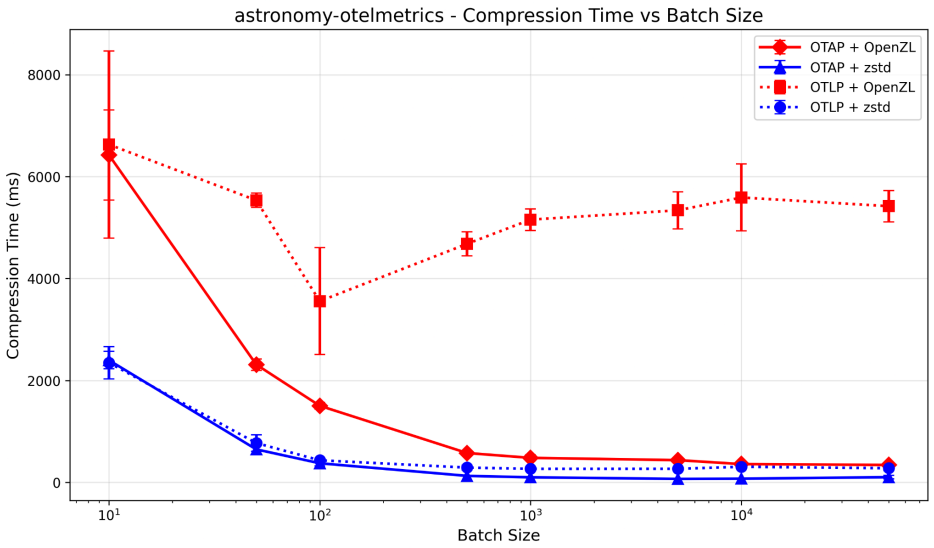
# Zstd level 4

## Compression Ratio




OpenZL significantly improves compression ratios for OTLP (row-based format), achieving substantially higher ratios compared to zstd alone.
For OTAP (columnar format), OpenZL provides minimal additional benefit over zstd, as OTAP's columnar layout already groups similar data together, reducing redundancy that OpenZL could exploit.

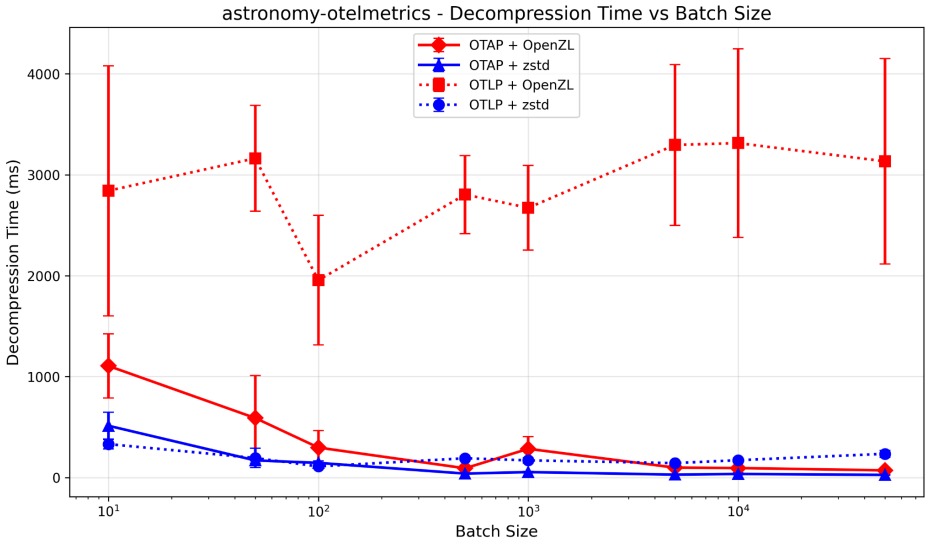
## Compression Time




OTLP + OpenZL consistently exhibits the highest compression time across most batch sizes.

At small batch sizes, OTAP + OpenZL incurs significant overhead. However, as batch size increases, its compression time converges toward that of zstd alone.

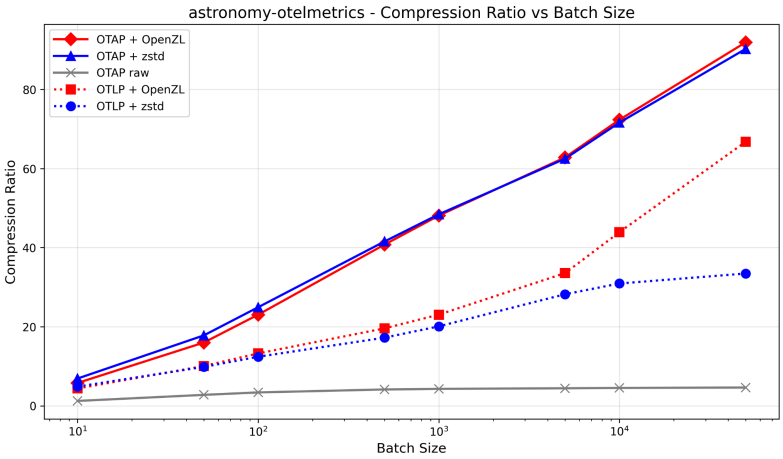
### Decompression Time




Decompression time follows a similar pattern.

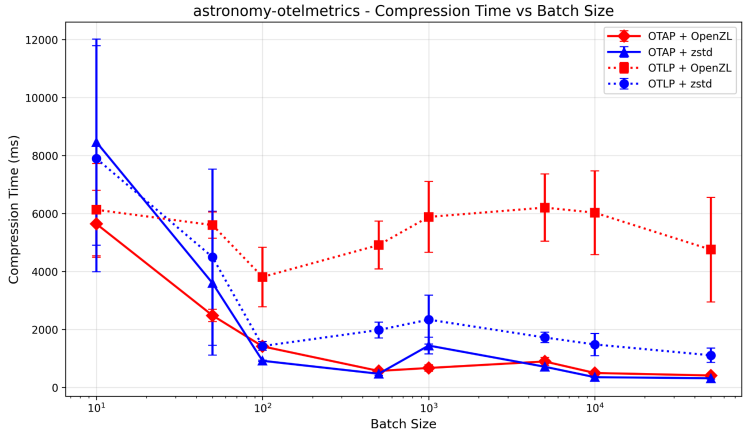
## Zstd level 7

### Compression Ratio



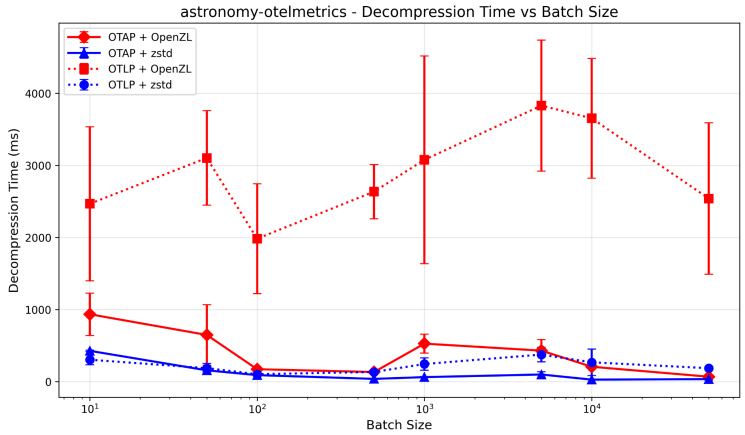

OpenZL provides negligible improvement for OTAP compression ratios at this level, as zstd's higher compression effort captures most available redundancy. OpenZL continues to substantially improve OTLP compression ratios compared to zstd alone.

## Compression Time




OTLP + OpenZL remains the slowest across most batch sizes.
Zstd compression times increase noticeably compared to level 4, shifting the zstd lines upward.

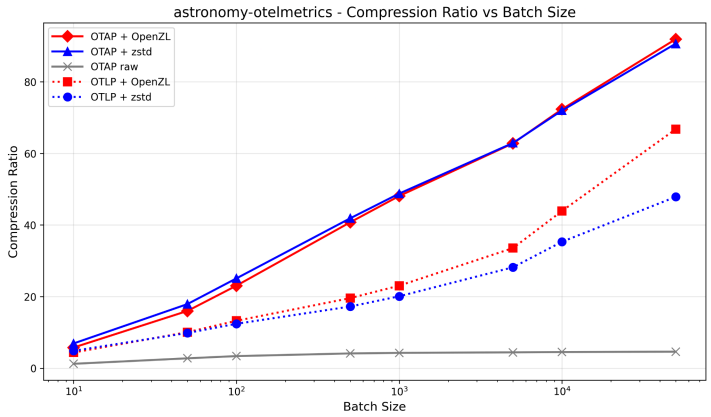
## Decompression Time



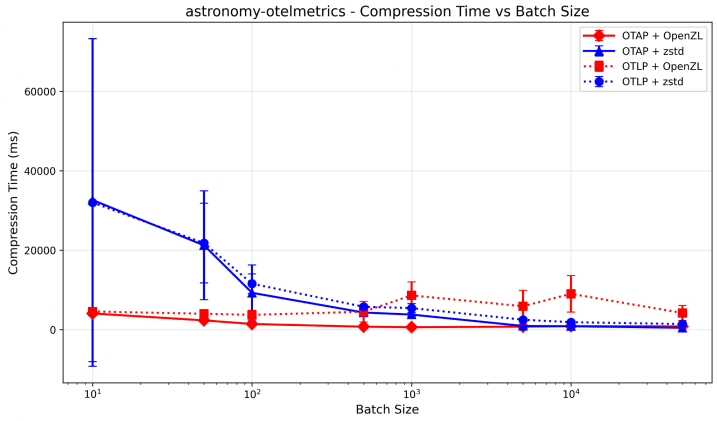

Decompression times remain largely unchanged from level 4.

# Zstd level 9

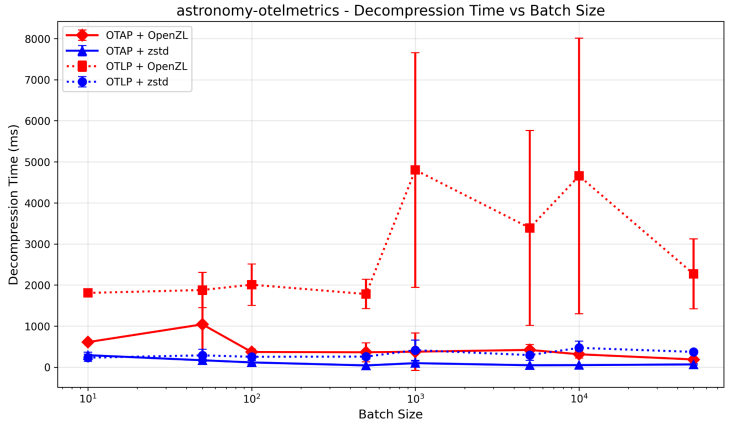
## Compression Ratio




## Compression Time



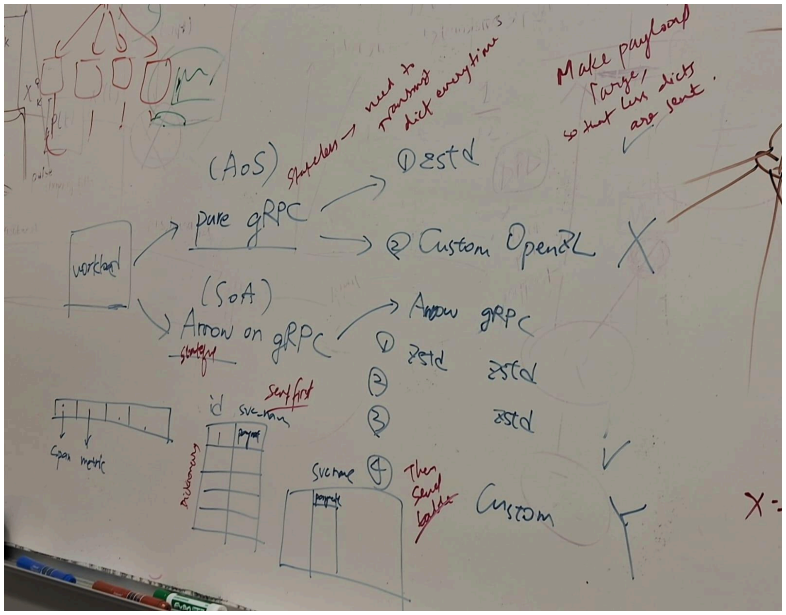

## Decompression Time



# 💬 Dave feedback

**First meeting**

- motivating the importance of the problem - large datacenter enterprise - cpu power and network bandwidth does otel take? fleet wide optimization percentage? are you running on only 1000 machines of a million?
- without apache arrow, maybe openzl can do better-- more interesting.... **another system that switched to apache arrow for IPC** - is a format specific compressor good enough and you don't need to change your IPC protocols? only evaluating otel with apache arrow - unconvincing job to answer the question. two systems - more progress.
- do they complement each other or do they obviate the need for each other? using a format specific compressor vs changing the whole format.... some systems that have made the switch to evaluate.
- more citations - be more clear about state of the art.
- **existence proof - otel people decided to switch to column oriented protocol- we can accept their reasons for that - self motivates the switch to columnar protocol.. Now we are opposing that and saying - retain your row oriented protocol - no need to switch, coz we have a new format aware compressor!**



# ⚙️ Alternate systems

## "Can a format-specific compressor (like OpenZL) make the switch to Apache Arrow unnecessary?"

**Stick with Dremio + one other**:

1. **Dremio** (strongest case - clear ODBC/JDBC → Arrow Flight migration for analytical queries) and **second case** : using ADBC.
2. **Apache Spark/PySpark** ( pickle → Arrow for JVM-Python IPC) or **Influx DB**
3. **OTEL-Arrow** (your original - OTLP → OTAP for telemetry)

This gives you diverse domains:

**Analytical queries** (Dremio)

**Distributed compute** (Spark)

**Observability** (OTEL)

**Timeseries** (InfluxDB)

All these have clear "we switched to columnar for performance" narratives that you can challenge with "format-aware compression on the original protocol could have been enough."

# Dremio

a) ODBC/JDBC (row oriented)
b) Arrow flight (arrow IPC over grpc)

| Workload | Protocol | Compressor | Compression Rate |
|---|---|---|---|
| Dremio | ODBC/JDBC | zstd | A |
| Dremio | ODBC/JDBC | OpenZL | B |
| Dremio | Arrow IPC on gRPC (Arrow flight) | zstd | C |
| Dremio | Arrow IPC on gRPC (Arrow flight) | OpenZL | D |

- B > A (OpenZL > zstd on row-oriented ODBC)
- D > C (OpenZL > zstd on columnar Arrow Flight)
- C > A (Columnar Arrow Flight > row-oriented ODBC)
- B ≈ D (Format-aware OpenZL is insensitive to row vs columnar format)

Yes, **Dremio does support ADBC** (Arrow Database Connectivity), and this adds an interesting layer to your research question.

# What is ADBC?

ADBC is a standard C API with existing implementations in C, C++, Go, Java, Python, R, C#/.NET, and Ruby, that can provide end-to-end columnar access Apache Arrow. Think of it as "JDBC/ODBC but for Arrow" - it's a standardized way to connect to databases using Arrow's native columnar format.

# Dremio's ADBC Support

Dremio supports ADBC through its Flight SQL interface, and as of May 2025, Microsoft Power BI has added an option to use the ADBC driver when connecting to Dremio Apache ArrowApache Arrow. The ADBC driver improves performance especially for large result sets Apache Arrow.

# How This Fits Your Research

This actually makes your research question **MORE interesting**! Here's the evolution:

### The Connection Type Hierarchy for Dremio:

1. **Traditional (Row-oriented)**
    - ODBC (row-based)
    - JDBC (row-based)
2. **Arrow-native (Columnar) - Multiple flavors**
    - **Arrow Flight** (Arrow IPC over gRPC) - Direct protocol
    - **Arrow Flight SQL ODBC** (Arrow Flight exposed through ODBC API)
    - **Arrow Flight SQL JDBC** (Arrow Flight exposed through JDBC API)
    - **ADBC** (Native Arrow database connectivity)

### Your Experimental Matrix Can Now Be:

| Protocol | Format | Compressor | Performance |
|---|---|---|---|
| ODBC (traditional) | Row-based | zstd | A |
| ODBC (traditional) | Row-based | **OpenZL** | B |
| Arrow Flight | Columnar | zstd | C |
| Arrow Flight | Columnar | **OpenZL** | D |
| **ADBC** | **Columnar** | zstd | E |
| **ADBC** | **Columnar** | **OpenZL** | F |

### The Key Research Question:

**Can OpenZL on traditional ODBC (B) match or beat Arrow Flight/ADBC with standard compression (C/E)?**

If **B ≈ C/E**, then your argument is: *"Format-specific compression eliminates the need for protocol migration to columnar formats"*

## Why ADBC Matters Here

ADBC was created because initially, the Arrow ecosystem used a mix of custom wire protocols and lacked a standard database interface, forcing users to use less performant row-based connectors and either experience the cost of transposing data or build custom database connectors Apache Arrow.

But your counter-argument could be: **"What if intelligent compression on row-based ODBC/JDBC could have been enough? Did we need a whole new standard?"**

## Bottom Line

**Yes, Dremio supports ADBC**, and this gives you an even richer landscape to evaluate:

- Traditional row-based (ODBC/JDBC)
- Columnar with custom protocol (Arrow Flight)
- Columnar with standard API (ADBC)

All three can serve as comparison points for your OpenZL compression approach!

## Apache Spark / PySpark (Major Production Switch)

Apache Spark integrated Arrow around 2016-2017 to dramatically improve PySpark performance by switching from pickle-based row-at-a-time serialization to Arrow's columnar block-by-block format Medium  GitHub. This was a fundamental architectural change:

**What they switched from:** Row-based pickle serialization between JVM and Python **What they switched to:** Arrow columnar format for IPC between Spark (JVM) and PySpark **Key improvements:**

- Eliminated high overhead in serialization/deserialization Medium
- Arrow-optimized Python UDFs execute ~1.9 times faster than pickled Python UDFs on 32 GB datasets Databricks
- Enabled zero-copy operations and direct use of Arrow's columnar format without deserialization Medium

**Your research angle:** This is perfect because Spark made the switch specifically to move to columnar format for better performance. You can argue: "What if OpenZL on the original pickle format could achieve similar compression ratios? Would the switch still be necessary?"

## InfluxDB 3.0 (Complete Rewrite Around Arrow)

InfluxDB 3.0 adopted Apache Arrow Flight as their network protocol, replacing their custom protocol from previous versions InfluxData. This was a complete architectural shift:

**What they switched from:** Custom TSM (Time-Structured Merge tree) storage engine with proprietary IPC **What they switched to:** Apache Arrow Flight for communication both within the cluster and to/from clients, using gRPC-based API to send Arrow arrays across the network InfluxData **Key benefits they cite:**

- Avoided defining their own efficient network protocol and implementing it in multiple languages (Rust, Golang, Python, Java) InfluxData
- Adding new language clients became straightforward by building on top of existing Flight clients InfluxData
- Zero-copy streaming of data

**Your research angle:** InfluxDB explicitly chose Arrow to avoid building custom serialization. Your counter-argument: "Could a format-aware compressor like OpenZL on their original TSM format have delivered comparable benefits without the protocol switch?"

# Your Research Framing

These two cases give you excellent fodder for your argument:

**The Hypothesis to Test:**

1. **Systems switched to Arrow primarily for:** Better compression, zero-copy operations, columnar layout advantages
2. **Your counter-claim:** A sophisticated format-specific compressor (OpenZL) applied to the *original* row-oriented protocols could achieve similar or better results without requiring:
    - Complete protocol rewrites
    - Migration overhead
    - Loss of existing optimizations in row-oriented formats

**Research Questions You Can Now Address:**

- Do Arrow's columnar benefits *complement* compression (both needed) or does good compression *obviate* the need for format changes?
- With OTEL-Arrow as case #1, Spark as case #2, and InfluxDB as case #3, can you show that format-aware compression provides comparable benefits without protocol migration?
- The existence proof: OTEL, Spark, and InfluxDB decided columnar was worth it - but were they lacking access to advanced format-specific compressors?

—----------

What doesn't work.

# Clickhouse native protocol

c) Row oriented - clickhouse http interface with various formats (JSON, CSV)
d) Columnar protocol - ClickHouse Native TCP protocol - a binary columnar format designed for efficient network transport and minimal server-side processing

**Different use cases**: The HTTP interface (JSON/CSV) is typically for human/scripting use, while the Native TCP protocol is for client libraries and inter-server communication
**Not really a migration**: ClickHouse has always had its Native protocol - they didn't "switch" from row to columnar for the same workload
**Apples to oranges**: Comparing HTTP/JSON to Native TCP mixes too many variables (HTTP overhead, JSON parsing, text vs binary)

# Apache kafka

e) Avro(row oriented) is widely used in Apache Kafka for efficient event serialization in real-time data streaming platforms
f) Parquet format (columnar)

This doesn't fit well because:

**Different purposes**: Avro is for streaming/messaging (low-latency individual events), Parquet is for storage/batch analytics (high-throughput columnar scans)

**Not IPC protocols**: Neither is really used for inter-process communication in the way OTEL, Dremio, or Spark use Arrow

**No migration story**: Systems don't typically switch from Avro to Parquet for the *same* workload - they use both for different purposes

# Clickhouse read/write – avro vs arrow

### Why This Doesn't Fit Your Research:

**This is NOT a protocol migration story**—it's just ClickHouse being format-agnostic. The actual communication happens through:

1. **ClickHouse Native TCP Protocol** (columnar, binary) - the primary high-performance protocol
2. **HTTP interface** (can return data in JSON, CSV, Avro, Arrow, or dozens of other formats)
3. **gRPC interface** (can also use various formats including ArrowStream)

So when you use Avro vs Arrow with ClickHouse, you're just choosing which **serialization format** to use over the same transport (HTTP/gRPC/file). There's no "we migrated from row-oriented to columnar" narrative.

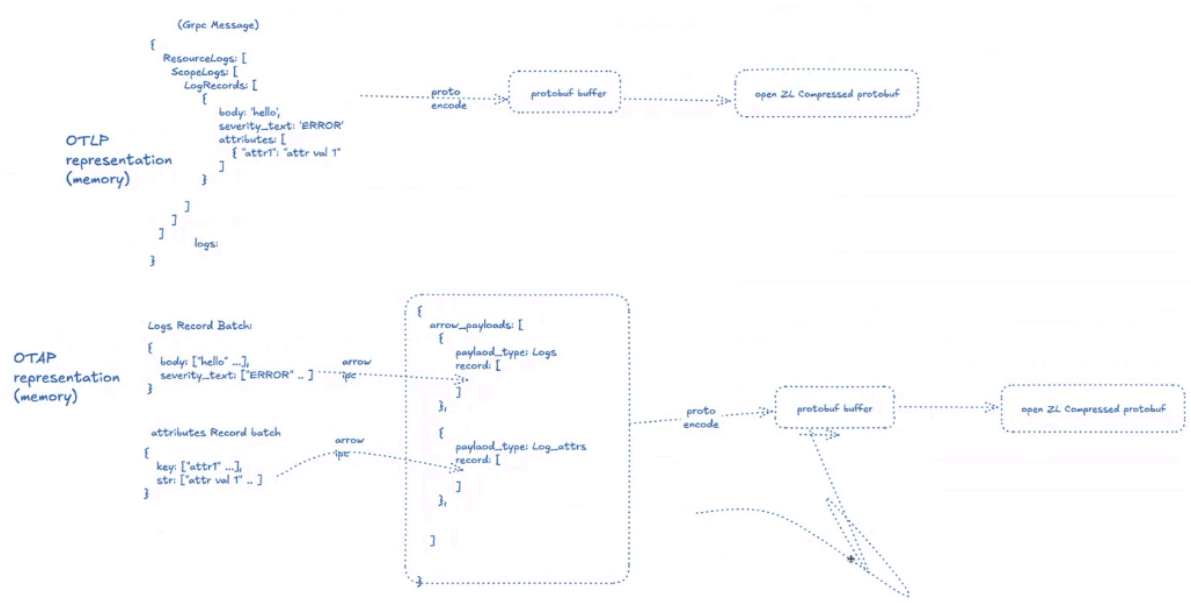
# Verdict for Your Research:

**ClickHouse Avro vs Arrow does NOT fit your problem** because:

1. ❌ **No migration story**: They coexist as format options, not a before/after
2. ❌ **Same transport**: Both use the same HTTP/gRPC protocols underneath
3. ❌ **Not an IPC protocol change**: Just different serialization formats for the same API

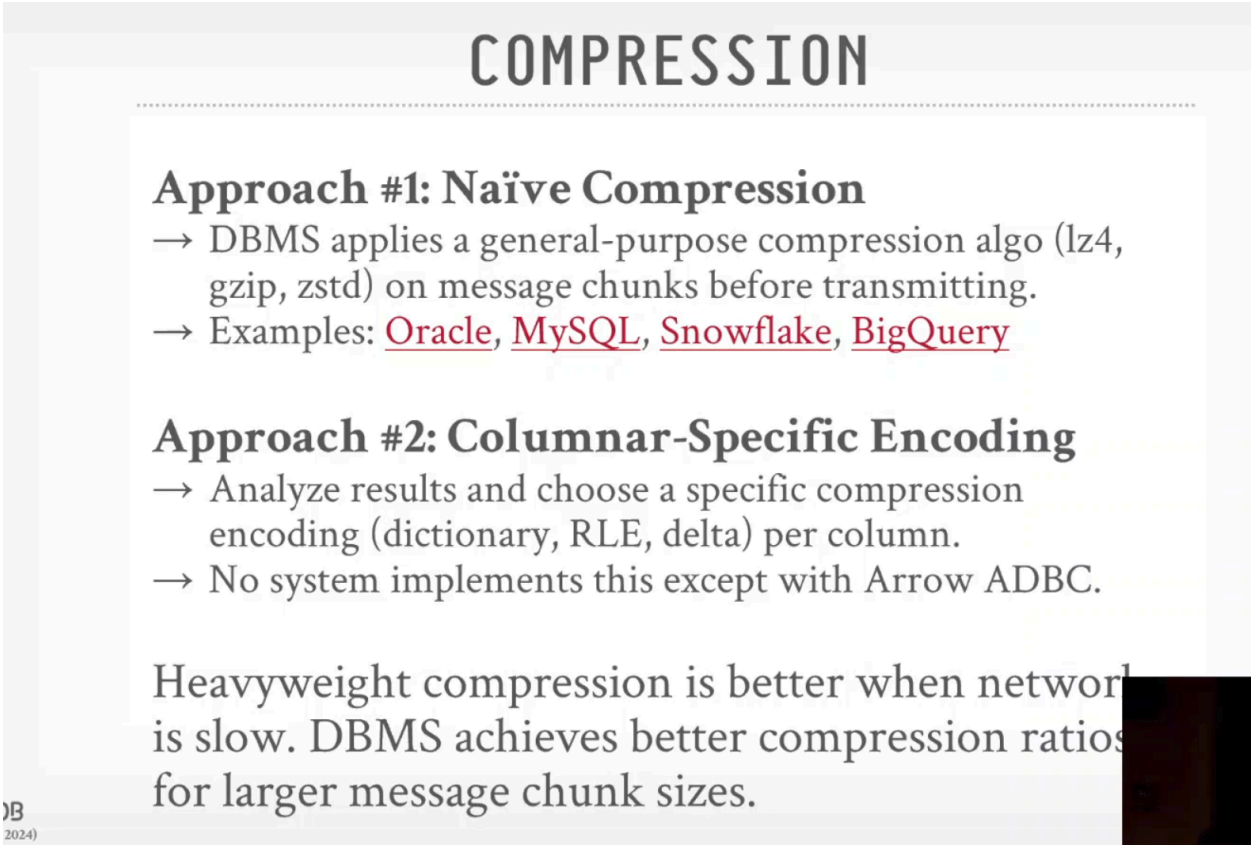
# 📒 Compression Pipeline

https://excalidraw.com/#json=q23d6De3BiGZahVCmPPYc,R-JEcuBXVLQ26JRmc0kXow




ADBC vs JDBC

S2024 #12 - Database Networking Protocols (CMU Advanced Database Systems)




Message chunk size

Apache Arrow ADBC
https://www.youtube.com/watch?v=TjlmNGNx77E&list=PLSE8ODhjZXjbEeW_bOCZ8c_nx_Jhoz-GW&index=8

A repo for evaluation:
https://github.com/splunk/stef/tree/d16e0a0aaf3a7f5798459505b2d2f2ff283e48b8?tab=readme-ov-file

# 🤔 Pragna Knowledge Doc

## OLTP very good blog -
## https://www.dash0.com/knowledge/opentelemetry-protocol-otlp

With OTLP, you can instrument your code once using standard OpenTelemetry SDKs and then transmit the data to any OTLP-compliant backend, whether commercial or open-source. Changing vendors now becomes a simple configuration tweak, not a major engineering project.

The OpenTelemetry Protocol provides the standardized format and transport that makes this portability possible. It defines a consistent format and set of rules for sending traces, metrics, and logs from a source (like your application) to a receiver (like the Collector or an observability backend).

**How traces are carried in OLTP -**
A trace is the end-to-end journey of a request through a distributed system. In OTLP, that story is told through spans, with each one representing a single operation, like a database query, an HTTP request, or a function call. Once they're linked together, you'll get a tree (or timeline) that shows the full execution path.
Spans are wrapped in a three-level envelope that mirrors the Resource → Scope → Signal hierarchy mentioned earlier. Even a single span is carried this way, though batches are common (and desired):

- resourceSpans: Groups spans by the resource that produced them.
- scopeSpans: Groups spans by the instrumentation scope.
- spans: One or more actual spans, as defined above.

**How metrics are represented in OLTP-**
OTLP also defines a structured and extensible data model for metrics, which is designed to align with existing ecosystems like Prometheus while adding richer semantics and cross-tool interoperability.Like traces, metrics are wrapped in a Resource → Scope → Signal  envelope.

**How logs are represented in OLTP -**
Log comes in many shapes and formats: plain text lines written by legacy applications and frameworks, structured JSON emitted by modern services, and system logs from the OS, containers, and infrastructure. Each carries valuable context, but differences in structure and semantics make it hard to aggregate and correlate them.

The OTLP log data model addresses this by:

- Normalizing log content with a common LogRecord schema that can represent both simple strings and rich structured objects.
- Preserving metadata from the original source via attributes
- Attaching correlation identifiers (trace_id, span_id) so logs can be automatically linked to distributed traces.

## Docs - https://opentelemetry.io/docs/specs/otlp/

In practice, using OTLP comes down to three key steps: instrumenting your applications, normalizing other formats through the Collector, and choosing a backend that can make full use of the data.

So far we've looked at what OTLP is and the structure of the data. But how does that data actually travel from your app to a backend?

OTLP is a request/response protocol. A client sends an Export*ServiceRequest and the server (collector, proxy, or backend) replies with a corresponding Export*ServiceResponse. This acknowledgment is the foundation of OTLP's hop-to-hop reliability since every handoff confirms whether the data was accepted or not.

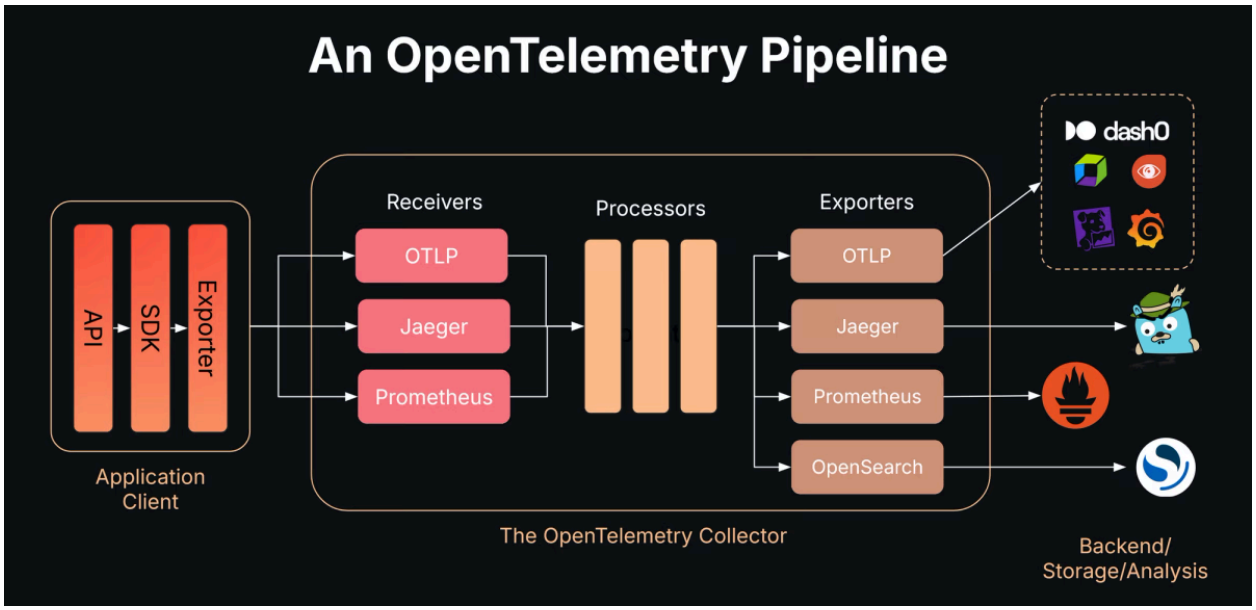
OTLP defines the encoding of telemetry data and the protocol used to exchange data between the client and the server.

This specification defines how OTLP is implemented over gRPC and HTTP transports and specifies Protocol Buffers schema that is used for the payloads.

Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data.

It’s like JSON, except it’s smaller and faster, and it generates native language bindings. You define how you want your data to be structured once, then you can use special generated source code to easily write and read your structured data to and from a variety of data streams and using a variety of languages.

Protocol buffers are a combination of the definition language (created in .proto files), the code that the proto compiler generates to interface with data, language-specific runtime libraries, the serialization format for data that is written to a file (or sent across a network connection), and the serialized data.




While it's possible to send OTLP data directly from your applications to a backend, in any production environment you should use the OpenTelemetry Collector.

The Collector provides the operational features that SDK exporters alone cannot: buffering, batching, enrichment, routing, and graceful handling of failures. These capabilities make the Collector the foundation of any production-grade OTLP deployment.

At its core, the Collector centralizes buffering and retry logic, so that transient failures or downstream outages don't immediately translate into dropped data.

It can also batch telemetry intelligently, combining small trickles of signals from many services into larger, more efficient payloads that reduce overhead and improve throughput.

Beyond transport, the Collector is capable of processing and enriching data—for example, attaching Kubernetes metadata, filtering out noisy telemetry, or applying advanced techniques like tail-based sampling to reduce cost while keeping the most valuable traces.

Just as importantly, it serves as a stable point of decoupling since applications only need to know how to reach the Collector, while backend changes can be managed entirely through configuration.

Taken together, these features make the Collector the operational backbone of any serious OTLP deployment.

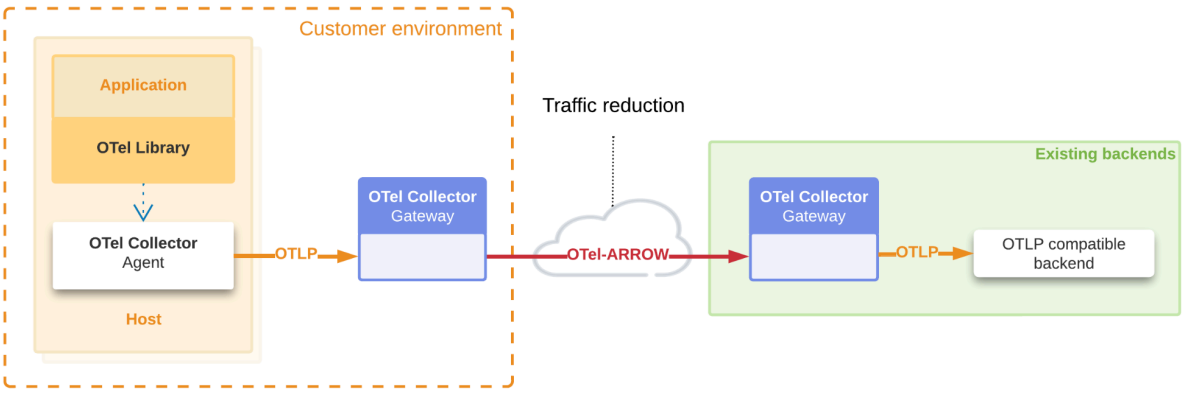
# OTAP OpenTelemetry Protocol with Apache Arrow (OTAP)

| Workload | Protocol | Compressor | Compression Rate |
|---|---|---|---|
| OpenTelemetry | Pure gRPC (OTLP) | zstd | A |
| OpenTelemetry | Pure gRPC (OTLP) | OpenZL | B |
| OpenTelemetry | Arrow IPC on gRPC (OTAP) | zstd | C |
| OpenTelemetry | Arrow IPC on gRPC (OTAP) | OpenZL | D |

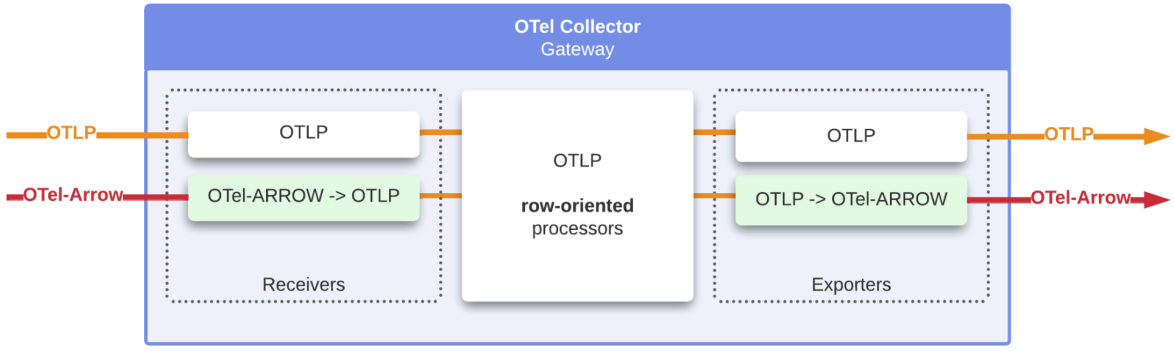
**OTel people used to use A.**
**Then they decided to switch to C : https://github.com/open-telemetry/otel-arrow**
**We want to show B is as good as D.**

## This is phase 1.
## project pitch is against double compression, but mostly it should be against switching to columnar protocol! And the logical worldly reason is that dont put effort to transform / switch if your target is compression rate.

Problem being -
“Current implementations employ a double compression architecture : individual Arrow IPC record batches are compressed with zstd, then the aggregated message is recompressed at the gRPC transport layer.”



The OpenTelemetry Collector-Contrib distribution includes the OTel-Arrow Receiver and Exporter. The following diagram is an overview of this integration, which supports seamless fallback from OTAP to OTLP. In this first phase, the internal representation of the telemetry data is still fundamentally row-oriented.




## Phase 2

We are building an end-to-end OpenTelemetry Protocol with Apache Arrow (OTAP) pipeline and we believe this form of pipeline will have substantially lower overhead than a row-oriented architecture.[coz apache arrow has good properties like zero copy data processing] See our Phase 2 OTAP Dataflow engine documentation.

These are our future milestones for OpenTelemetry and Apache Arrow integration:

1. Extend OpenTelemetry client SDKs to natively support the OpenTelemetry Protocol with Apache Arrow Protocol
2. Extend the OpenTelemetry collector with direct support for OpenTelemetry Protocol with Apache Arrow pipelines
3. Extend OpenTelemetry data model with native support for multi-variate metrics.
4. Output OpenTelemetry data to the Parquet file format, part of the Apache Arrow ecosystem

ADBC

https://medium.com/@tfmv/the-arrow-that-binds-5e6adfbb917d

https://arrow.apache.org/blog/2023/01/05/introducing-arrow-adbc/

https://voltrondata.com/blog/simplifying-database-connectivity-with-arrow-flight-sql-and-adbc

**Arrow Database Connectivity, a new standard for the database interface layer.**

Launched in early 2023 by the Apache Arrow community, ADBC is neither a wrapper nor a driver. It is a declaration: that the interface between databases and applications should be fast, frictionless, and faithful to the shape of the data.

Where ODBC and JDBC shuttle data in rows and require format conversion at both ends, ADBC is Arrow-native by design. It streams RecordBatches, not rows. It preserves schema, types, and structure across boundaries. It avoids copies, embraces streaming, and speaks in a dialect analytics engines understand.

It is, quite simply, a modern API standard for high-performance analytical connectivity. Not a protocol, not an SDK, but the language glue that binds systems together.

# Evaluation procedure

1) Procure a dataset for that workload - containing payloads (traces, logs, metrics in OTel) – gRPC payloads.

https://data.cityofnewyork.us/Transportation/2020-Green-Taxi-Trip-Data/pkmi-4kfn/about_data

**Their native (row) format with OpenZL**

1) Compile OpenZL. - https://openzl.org/getting-started/quick-start/

2) Implement OpenZL –
    a) Parser for OTAP. write parser for your workload. https://github.com/danielthank/otel-arrow/blob/openzl/openzl-experiment/cpp/openzl-arrow-parser/otap_parser.cpp
    b) Train function - probably this can be reused. Register above parser in the training program. https://github.com/danielthank/otel-arrow/blob/openzl/openzl-experiment/cpp/openzl-arrow-parser/train_otap.cpp
    c) Use this parser and train it with the collected dataset as input and try to output a compressor for this task.
3) Use the OpenZL compressor to compress the above payload – this is also just calling their API with payload and the compressor as input, and get the output
4) Use this output to get compression ratio

**The other step is to get compression ratio for their columnar format with zstd (or whatever their compressor is) -**

1) Needs a different payload - columnar payload — representing the same thing as above : use STEF to transform row to columnar.
2) just call zstd function - basically replace compressor in step 3.

# pkl vs arrow workloads

Quick reminders and next steps can use later

- Scripts I added (all under scripts):
  - `run_capture.py` — capture pickle + Arrow payloads from a CSV
  - `run_train_openzl.py` — train OpenZL (wraps `zli train`)
  - `train_zstd_dictionary.py` — train zstd dictionary (can split/replicate samples)
  - `compress_and_compare.py` — compress with OpenZL and zstd+dict and write JSON summary

To rerun on the full dataset (example commands)

- Put the CSV at `openzl/data/yellow_tripdata_2019-01.csv` (or provide an accessible URL). Then:

# 1) Capture payloads (pickles + Arrow)

python3 openzl/experiments/scripts/run_capture.py --csv
openzl/data/yellow_tripdata_2019-01.csv

# 2) Train OpenZL on pickles

python3 openzl/experiments/scripts/run_train_openzl.py --samples-dir
openzl/experiments/payloads/pickle --out openzl/experiments/trained_openzl.zc

# 3) Train zstd dictionary on Arrow payloads

#  For a fair run, do NOT use --replicate if you have lots of samples.

python3 openzl/experiments/scripts/train_zstd_dictionary.py --samples-dir
openzl/experiments/payloads/arrow --out openzl/experiments/arrow_dict.zstdict --replicate

# 4) Compress and compare

python3 openzl/experiments/scripts/compress_and_compare.py \

  --pickle-dir openzl/experiments/payloads/pickle \

  --arrow-dir openzl/experiments/payloads/arrow \

  --openzl-compressor openzl/experiments/trained_openzl.zc \

```
  --zstd-dict openzl/experiments/arrow_dict.zstdict \
  --out openzl/experiments/compare_summary.json
```

**A couple of brief tips**

For a fair zstd comparison, avoid --replicate and ensure the zstd trainer sees many genuine small samples (splitting large Arrow files is OK, but synthetic replication skews results).

Make sure zli is built (check openzl/cachedObjs/*/zli) or available on PATH.

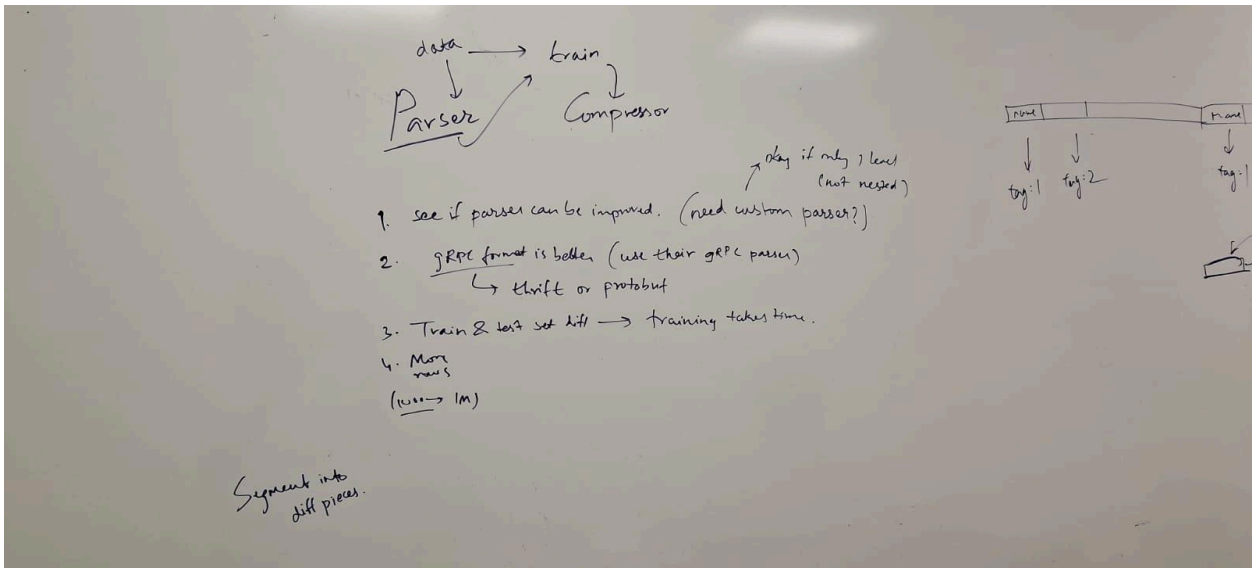
If you want, I can: (a) run the pipeline for you when you upload the big CSV or share a working URL, (b) run without replication, and (c) produce a short report summarizing per-file and aggregate results.

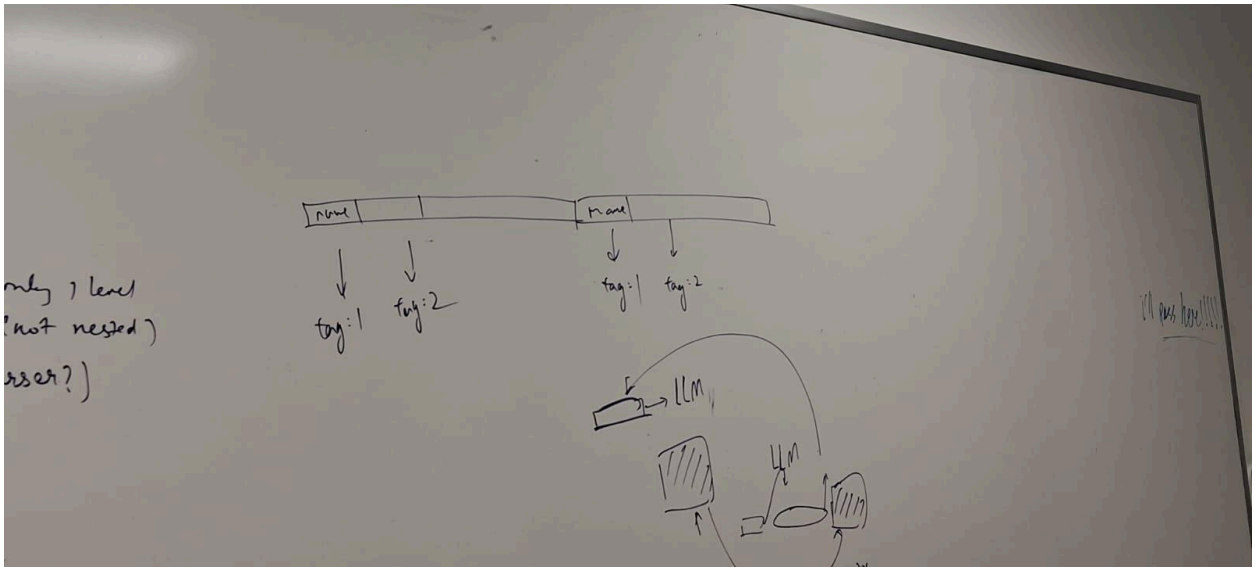
# Pragna ToDos

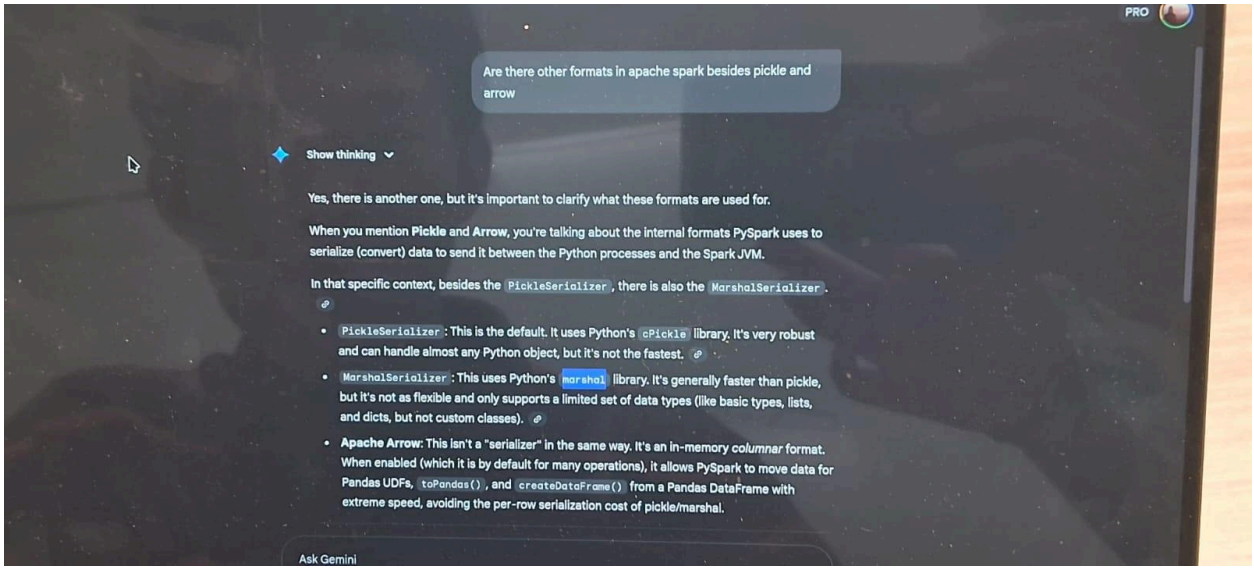
TPC-H - more academic

Put in grpc format for doing the row based experiments. Less risky.
Column is anyway arrow.

Less time – so limit scope to just gRPC vs arrow.




PRO
Are there other formats in apache spark besides pickle and arrow
Show thinking
Yes, there is another one, but it's important to clarify what these formats are used for.
When you mention Pickle and Arrow, you're talking about the internal formats PySpark uses to serialize (convert) data to send it between the Python processes and the Spark JVM.
In that specific context, besides the PickleSerializer, there is also the MarshalSerializer.
PickleSerializer: This is the default. It uses Python's cPickle library. It's very robust and can handle almost any Python object, but it's not the fastest.
MarshalSerializer: This uses Python's marshal library. It's generally faster than pickle, but it's not as flexible and only supports a limited set of data types (like basic types, lists, and dicts, but not custom classes).
Apache Arrow: This isn't a "serializer" in the same way. It's an in-memory columnar format. When enabled (which it is by default for many operations), it allows PySpark to move data for Pandas UDFs, toPandas(), and createDataFrame() from a Pandas DataFrame with extreme speed, avoiding the per-row serialization cost of pickle/marshal.
Ask Gemini